PhysioGuide: A Technical Blueprint for MVP Development

Part I: Foundational Knowledge & Resource Compendium

This initial section establishes the conceptual and technical groundwork for the project. Its purpose is to onboard the entire team to a shared understanding of the technologies, principles, and inherent challenges of the PhysioGuide system. A unified knowledge base is the first step toward mitigating integration risks and ensuring that each specialist's contribution aligns with the project's overarching goals.

1.1 The PhysioGuide Ecosystem: A Technology Stack Overview

The PhysioGuide system is best understood not as a collection of disparate technologies, but as a cohesive, real-time data pipeline.[1] This pipeline begins with visual data capture, proceeds through intelligent processing, and culminates in an immersive user experience. The architecture can be divided into three core pillars: Perception (OpenPose), Logic & Communication (Python), and Presentation (Unity/VR).

The Perception Layer (OpenPose)

At the front end of the pipeline is the perception layer, powered by OpenPose. OpenPose is a real-time, multi-person keypoint detection library that uses advanced neural networks to identify human body, hand, and facial keypoints from visual inputs like webcams, videos, or static images.[4] For the PhysioGuide project, its primary function is to analyze the video stream from a standard webcam and output the 2D coordinates of a user's anatomical landmarks, forming a digital skeleton.[1] The project will leverage the BODY_25 model, which provides 25 keypoints covering the major joints and body parts, offering a good balance between detail and performance.[7] While the core OpenPose library is written in C++, the project will utilize its official Python API, which allows for more rapid development and easier integration with other Python libraries for data processing and communication.[9]

The Logic & Communication Layer (Python)

Python serves as the central nervous system of the application. Its role extends far beyond simply executing the OpenPose model. The Python backend is responsible for orchestrating the flow of data and embedding the system's core intelligence.[1] This layer performs several critical functions:

1. **Data Ingestion:** It captures the raw video feed from the webcam using libraries like OpenCV.
2. **Pose Processing:** It passes each frame to the OpenPose Python wrapper to get the raw keypoint data.
3. **Biomechanical Analysis:** It interprets the raw (x, y) coordinates, validates them using confidence scores, and calculates biomechanically relevant metrics, such as the joint angles required for exercise analysis.
4. **Real-time Broadcasting:** It serializes the processed data into a structured format and broadcasts it over the network to the presentation layer.

The Presentation Layer (Unity & Oculus Quest 2)

The presentation layer is the immersive front-end where the user interacts with the system and receives feedback.[1] This layer is built using the Unity game engine, a powerful platform for creating real-time 3D and VR experiences. The target hardware, the Oculus Quest 2, is a standalone VR

headset that runs on an Android-based operating system. This is a crucial technical detail, as it mandates that the Unity project must be configured with "Android Build Support" to create applications that can run on the device.[1] The Unity application will receive the pose data from the Python backend, use it to animate a virtual avatar, apply the rehabilitation rules, and render the appropriate visual and audio feedback to the user in the virtual environment.

Critical Limitation Analysis: 2D vs. 3D Pose Estimation

A core architectural decision in the project proposal is the use of a single 2D webcam for pose estimation.[1] While OpenPose is capable of 3D reconstruction with multi-camera setups, the MVP's reliance on a single camera introduces a significant challenge: perspective distortion.[5] A 2D camera provides a flat projection of a 3D movement. An angle, such as shoulder abduction, will be measured differently depending on the user's orientation and distance relative to the camera. For example, if a user performs a 90-degree shoulder abduction, the angle measured in the 2D image will only be accurate if their torso is perfectly parallel to the camera's image plane. Any rotation of the body will cause foreshortening, leading to inaccurate angle calculations.

This is arguably the greatest source of potential error in the MVP. The system's robustness will depend on controlling this variable. The initial software design must include clear on-screen instructions for the user, guiding them to stand in a specific, marked position and maintain a consistent frontal orientation to the camera. Furthermore, this limitation must be explicitly documented in the project's final report. The system architecture should be designed with modularity in mind, allowing for a future upgrade path to incorporate 3D depth sensors (e.g., Intel RealSense, Azure Kinect) or a multi-camera rig, which would provide true 3D keypoint data and eliminate the perspective problem entirely.

1.2 Core Concepts in Computer Vision for Rehabilitation

To effectively build and debug the PhysioGuide system, the team must understand the fundamental principles of the computer vision pipeline.

From Pixels to Skeletons

The process of pose estimation is not magic; it is a sophisticated application of deep learning. The Convolutional Neural Network (CNN) at the heart of OpenPose analyzes the input image through many layers to produce two key intermediate outputs: Part Confidence Maps and Part Affinity Fields (PAFs).[4]

- **Part Confidence Maps:** These are a set of heatmaps, one for each body part (e.g., right elbow, left knee). The brightness of a pixel in a heatmap corresponds to the model's confidence that the specific body part is located at that pixel.
- **Part Affinity Fields:** These are vector fields that encode the direction and association between different body parts. For example, the PAF for the forearm would contain vectors pointing from the elbow to the wrist for every person in the image.

OpenPose then uses a greedy parsing algorithm to connect the high-confidence points from the maps using the guidance from the PAFs, assembling them into a complete skeleton for each person detected.

Understanding the Output Data

The final output from OpenPose for a single person is typically an array of keypoints, corresponding to a specific model like BODY_25.[7] Each keypoint in this array is represented by three values: its horizontal (x) coordinate, its vertical (y) coordinate within the 2D image frame, and a confidence score.

The Importance of Confidence Scores

The keypoint data generated by OpenPose is probabilistic, not deterministic. The confidence score, typically a value between 0.0 and 1.0, represents the model's certainty about the location of a specific keypoint. This is a critical piece of information that cannot be ignored.

A robust system must operate on the principle of "garbage-in, garbage-out." If the confidence score for a crucial joint, like an elbow or shoulder, is low due to factors like poor lighting, occlusion (e.g., the user's arm is hidden behind their body), or an unusual camera angle, then any joint angle calculated using that keypoint's position will be inherently unreliable. Therefore, the Python logic that processes the OpenPose data cannot blindly trust every coordinate it receives. It must implement a validation step using confidence score thresholding. For example, before calculating the elbow flexion angle, the system must check the confidence scores for the shoulder, elbow, and wrist keypoints. If any of these scores fall below a predefined threshold (e.g., 0.5), the data for that frame should be considered invalid. In such a case, the system should not provide corrective feedback but should instead instruct the user to improve their visibility to the camera, ensuring that the feedback loop is driven only by high-quality data.

Calculating Joint Angles from Keypoints

The translation of raw keypoint data into meaningful biomechanical information is a purely geometric task. To calculate the angle of a joint, three keypoints are required: the joint itself (the vertex of the angle) and the two adjacent keypoints that define the limbs. For example, to calculate the right elbow angle, the coordinates of the RShoulder, RElbow, and RWrist are needed.

The calculation involves these steps:

- Define Vectors: Create two vectors from the coordinates. The first vector, v 1, runs from the elbow to the shoulder. The second vector, v 2, runs from the elbow to the wrist.

v 1=RShoulder−RElbow

v 2=RWrist−RElbow

- Apply the Dot Product Formula: The angle θ between these two vectors can be found using the dot product formula:

$$\theta = \arccos(\|v1\|\|v2\| v1 \cdot v2)$$

This formula can be implemented efficiently in Python using a library like NumPy, which provides functions for vector operations, dot products, and calculating norms (magnitudes).

1.3 Principles of Immersive Application Development in Unity

This section provides an essential overview for the entire team, with a primary focus on the tasks assigned to the Unity & VR Developer.

The VR Development Toolchain

A stable and correctly configured development environment is non-negotiable. The required software components for this project are:

- **Unity Hub:** A management tool used to install and maintain multiple versions of the Unity Editor.[17]
- **Unity Editor (LTS Version):** The core development environment. Using a Long-Term Support (LTS) version is critical for project stability, as these versions receive bug fixes for an extended period without introducing breaking API changes.[1]
- **Meta Quest Developer Hub (ODH):** A utility provided by Meta for managing Quest devices, installing application builds (.apk files), and accessing debugging tools.[1]

The Meta XR SDK Suite

To enable VR functionality in Unity, the project relies on a suite of Software Development Kits (SDKs) provided by Meta. It is important to understand the role of each:

- **Meta XR Core SDK:** This is the foundational package. It provides the essential link between Unity and the Quest headset, including the core camera rig (OVRCameraRig or XROrigin), head and controller tracking, and the interface with the device's runtime.[17]
- **Meta XR Interaction SDK:** This is a higher-level SDK that provides pre-built components for common VR interactions. This includes scripts and prefabs for grabbing objects, poking buttons, and interacting with user interfaces.[20] Utilizing this SDK will significantly accelerate the development of interactive elements in the VR gym.

VR Scene Fundamentals

A VR scene in Unity differs from a traditional 3D game scene. The main camera is no longer a static object but is replaced by a special prefab, often called OVRCameraRig or XROrigin.[1] This prefab is the virtual representation of the user. It contains objects corresponding to the user's head (the headset) and hands (the controllers). At runtime, the Meta XR SDK automatically updates the position and rotation of these objects to match the user's real-world movements, creating the sense of immersion.

Performance is Paramount

Performance is the most critical technical constraint in VR development. A traditional PC or console game might be acceptable at 30 or 60 frames per second (FPS). In VR, a stable and high frame rate is required to prevent motion sickness (cybersickness). For the Oculus Quest 2, the target frame rate is typically 72 FPS or 90 FPS. If the application's performance drops below this target, the user will experience stuttering and visual artifacts that can be highly disorienting. This constraint must inform every development decision, from the complexity of the 3D models and textures used in the virtual gym to the efficiency of the C# scripts that process the incoming pose data.

1.4 Translating Biomechanics into Code: Engineering the Feedback System

This chapter bridges the gap between the theoretical research conducted by the R&D Specialist and the practical implementation of the feedback logic by the developers.

Defining "Correct Form" Algorithmically

For the purpose of the MVP, the concept of "correct exercise form" must be translated from qualitative descriptions into a set of deterministic, quantifiable, geometric rules.[1] These rules will be based on the joint angles and relative positions of keypoints derived from the OpenPose data. The feedback system will be a simple, rule-based engine that compares the user's real-time biomechanical data against these predefined rules.

Case Study: Engineering a Rule for Shoulder Abduction

The process of creating a rule for an exercise like shoulder abduction serves as a template for all other exercises. The steps are as follows, drawing on biomechanical research and the OpenPose keypoint model [7]:

1. **Identify Keypoints:** From the BODY_25 model, the necessary keypoints are RShoulder (index 2), RElbow (index 3), and RHip (index 8). The hip is used to establish a stable reference for the torso.
2. **Define Vectors:** Two vectors are defined in the 2D plane of the camera image. The first vector, v_torso, represents the orientation of the torso and is defined from the RShoulder keypoint to the RHip keypoint. The second vector, v_arm, represents the upper arm and is defined from the RShoulder to the RElbow.
3. **Calculate the Primary Angle:** The primary metric for shoulder abduction is the angle between v_torso and v_arm. This angle is calculated using the dot product formula previously described.
4. **Define the Correct Range of Motion (ROM):** Based on clinical sources, the full ROM for shoulder abduction can reach 180 degrees.[25] However, a therapeutic exercise might only require movement through a partial range, for example, from a resting position of 10 degrees up to a peak of 90 degrees for a single repetition.[27] This target range defines the "correct" movement.
5. **Detect Common Errors:** A simple angle check is insufficient for robust feedback. The system must also detect common mistakes. A frequent error in shoulder abduction is "shoulder hiking," where the patient elevates their shoulder by engaging the upper trapezius muscle instead of isolating the deltoid.[27] This can be detected algorithmically by monitoring the vertical (y) coordinate of the RShoulder keypoint. If this coordinate increases significantly relative to the torso length during the abduction phase, it indicates improper form, and specific feedback can be triggered. Another error is leaning the torso, which can be detected by monitoring the angle of v_torso relative to the vertical axis of the image.

The Exercise Specification Document

To formalize this process, the R&D specialist will create an "Exercise Specification Document" for each exercise. This document will be the definitive technical specification for the feedback logic, ensuring that the rules implemented by the developers are grounded in sound biomechanical principles. A template for this document will be provided in Part III.

1.5 The Data Bridge: Real-Time Inter-Process Communication

The most critical integration point in the PhysioGuide architecture is the communication channel between the Python backend and the Unity frontend.[1] This bridge must be low-latency, reliable, and easy to implement.

Protocol Selection: WebSockets vs. ZeroMQ

Two common technologies for this type of inter-process communication are WebSockets and ZeroMQ.

- **ZeroMQ** is a high-performance, asynchronous messaging library. It is extremely powerful and flexible, supporting complex messaging patterns like publish-subscribe, request-reply, and pipelines. However, its power comes with a steeper learning curve and more complex setup.[30]
- **WebSockets** provide a simpler, full-duplex communication channel over a single, persistent TCP connection. Originally designed for web browsers, they are now widely used for any real-time client-server communication. The protocol is well-supported with robust and easy-to-use libraries in both Python and C#.[31]

For the PhysioGuide project, the communication requirement is straightforward: a continuous, one-way stream of structured data from a single server (the Python application) to a single client (the Unity application). In this context, the complexity of ZeroMQ is unnecessary. WebSockets provide the ideal balance of performance and simplicity. Choosing the simpler, more direct tool is a significant de-risking strategy for a project with a tight deadline and a team new to these domains. Therefore, WebSockets are the recommended protocol.

Recommended Implementation

The recommended implementation uses a combination of modern, well-supported libraries:

- **Python Server:** The **FastAPI** web framework is recommended for the Python backend. FastAPI has native, high-performance support for WebSockets built upon the Starlette ASGI toolkit, making it simple to create a robust WebSocket server.[31]
- **Unity Client:** The **WebSocket-Sharp** library is a popular and reliable choice for implementing a WebSocket client in C# for Unity. It can be added to the project as a simple .dll file.[33]

Defining the Data Contract

To ensure seamless integration and enable parallel development, a strict data contract must be defined upfront. This contract specifies the exact structure of the data packets that will be sent from the server to the client. **JSON (JavaScript Object Notation)** is the recommended format due to its human-readability and excellent library support in both Python and C#. The Python server will serialize its processed data into a JSON string, and the Unity client will deserialize this string back into a C# object. The precise structure of this JSON object will be detailed in Part III.

The following table provides a single source of truth for all software dependencies. Adhering to these specific versions will prevent common and time-consuming version conflicts, ensuring that code developed on one team member's machine will function correctly on another's.

Table 1: Technology Stack & Versioning Control

Component	Software	Recommended Version	Justification / Compatibility Notes
VR Development	Unity Hub	3.x	For managing multiple Unity Editor installations.
	Unity Editor	**2022.3.x LTS**	Long-Term Support (LTS) version ensures stability. Required for recent Meta XR SDKs.[18]
	Meta XR Core SDK	Latest (via Package Manager)	Provides core VR functionality.[19]
	Meta XR Interaction SDK	Latest (via Package Manager)	Required for user interaction like grabbing.[20]
Pose Estimation	Python	**3.7.x**	Known compatibility with older OpenPose builds; check latest OpenPose docs. Avoid >3.8 initially.[36]
	OpenPose	Latest from GitHub Master	To get the latest features and bug fixes.
	Visual Studio	**2019 Community**	Required for compiling OpenPose from source on Windows.[1]
	CMake	Latest GUI version	For generating the Visual Studio solution for OpenPose.[1]
	NVIDIA CUDA Toolkit	Version specified by OpenPose	**Crucial:** Must match OpenPose requirements. Check their GitHub repo.
	NVIDIA cuDNN	Version specified by OpenPose	**Crucial:** Must match CUDA version and OpenPose requirements.

Component	Software	Recommended Version	Justification / Compatibility Notes
Communication	FastAPI (Python)	Latest (via pip)	Modern, high-performance Python web framework with WebSocket support.[31]
	WebSocket-Sharp (Unity)	Latest (via NuGet/dll)	Well-known and reliable WebSocket client for C#/.NET.[34]
Version Control	Git	Latest	Standard for source code management.

Part II: The Two-Week Foundation Sprint: An Accelerated Execution Plan

This section provides a granular, task-oriented execution plan for the project's initial two-week sprint. It expands upon the team's kickoff document by adding technical depth, defining clearer success criteria, and introducing critical tasks to de-risk the project's core technologies.[1]

2.1 Sprint Mandate and Success Criteria

Objective: The singular goal of this sprint is to eliminate foundational risks by establishing fully functional and validated development environments for each specialist. By the end of the two weeks, each team member must have successfully completed a "Hello, World" style proof-of-concept for their respective technology stack.

Overall Success Criterion: The sprint is considered successful when all four team members can demonstrate their individual deliverables as defined in the task matrix below. The project's GitHub repository must be active, containing the initial project structure and commits from each developer's feature branch. The first weekly sync meeting has been successfully conducted, and the project's progress is being tracked on a shared task board.

2.2 Detailed Task Matrix by Team Member

The following table outlines a day-by-day plan for the two-week sprint. This schedule is ambitious but achievable and is designed to build momentum and validate core assumptions early in the project lifecycle.

Table 2: Two-Week Sprint Task Matrix

Member	Day	Task Description	Key Resources / Actions	Deliverable / Success Criterion
Taha Khan	1-2	**Project Infrastructure Setup:** Create the private GitHub repository.	Use Git command line or a GUI client. Research	Repository created with correct branch

Member	Day	Task Description	Key Resources / Actions	Deliverable / Success Criterion
		Implement the GitFlow branching model (main, develop). Create initial feature branches: feature/vr-environment and feature/pose-estimation-module. Invite all team members as collaborators.	GitFlow branching strategy.	structure. All members have access.
(Project Lead)	3-4	**Documentation & Task Management:** Initialize the GitHub Wiki. Create pages for "Project Goals," "Architecture," and a placeholder for "Meeting Notes." Set up a GitHub Project board (Kanban style) and populate it with the tasks from this sprint plan, assigning each to the responsible individual.	Explore GitHub's Wiki and Projects features.	A functional Kanban board is live. The initial Wiki structure is in place.
	5-7	**Schedule & Communication:** Schedule the recurring weekly 1-hour sync meeting. Establish a primary communication channel. Conduct the first sync meeting to review Week 1 progress and confirm Week 2 goals.	Use a shared calendar (e.g., Google Calendar).	Weekly meeting is on the calendar. First meeting successfully held.
	8-14	**"Hello, Network" Integration PoC:** Develop a basic Python server using FastAPI that sends	FastAPI WebSocket docs.[31] WebSocket-Sharp docs.[34]	A standalone Python script successfully communicates

Member	Day	Task Description	Key Resources / Actions	Deliverable / Success Criterion
		a timestamp via WebSocket every second. Develop a basic Unity C# script (non-VR) using WebSocket-Sharp that connects to the server, receives the timestamp, and prints it to the Unity console.		with a running Unity Editor project.
Javeria Anwer	1-5	**Unity & VR Environment Setup:** Install Unity Hub and Unity Editor 2022.3.x LTS, ensuring "Android Build Support" is included.[11] Install the Meta Quest Developer Hub (ODH). Follow the official guide to enable Developer Mode on the Oculus Quest 2.[10]	Official Unity and Meta documentation.[17]	All software is installed. The Quest 2 is in Developer Mode and is recognized by the PC and ODH.
(Unity & VR Dev)	6-7	**Project Configuration:** Create a new 3D (URP) Unity project. Switch the build platform to Android/Meta Quest. Use the Package Manager to import the Meta XR Core SDK and Meta XR Interaction SDK.[20] Use the Project Setup Tool to apply all recommended settings.[18]	Unity Package Manager, Meta XR Project Setup Tool.	A new Unity project is correctly configured for Quest 2 development with all required SDKs installed.
	8-14	**"Hello, VR" Interaction Task:** Create a new scene. Add the OVRCameraRig prefab. Add a 3D cube to the scene. Add the	Meta XR Interaction SDK documentation.[21]	A running application on the Quest 2 where the user can see the virtual cube and

Member	Day	Task Description	Key Resources / Actions	Deliverable / Success Criterion
		GrabInteractable and Grabbable components to the cube. Build and run the scene on the Quest 2.		successfully grab and manipulate it with the controllers.
Hussain Kazmi	1-7	**OpenPose Prerequisites & Dependencies:** Install Visual Studio 2019 with the "Desktop development with C++" workload.[1] Install CMake GUI. **GPU Path:** Carefully identify, download, and install the specific versions of the NVIDIA CUDA Toolkit and cuDNN library required by the latest OpenPose build. Set system environment variables correctly.	OpenPose GitHub installation guide.[36] NVIDIA developer website.	All prerequisite software is installed. For GPU path, nvidia-smi command runs successfully and shows the correct CUDA version.
(OpenPose Spec.)	8-12	**OpenPose Compilation:** Clone the OpenPose repository and its submodules. Use CMake GUI to configure the project, pointing to the source and a new build directory. **Crucially, enable the BUILD_PYTHON flag.** If using CPU-only, set GPU_MODE to CPU_ONLY. Generate the Visual Studio solution. Open the .sln file and compile the entire solution in Release mode.	OpenPose compilation documentation.[36]	The OpenPose project compiles successfully in Visual Studio without any build errors. The pyopenpose module is generated in the build directory.

Member	Day	Task Description	Key Resources / Actions	Deliverable / Success Criterion
	13-14	**"Hello, Pose" Python Task:** Write a Python script that imports pyopenpose and OpenCV. The script must initialize the OpenPose wrapper, open a live webcam feed, process each frame, and display the resulting image with the detected skeleton overlaid. It must also print the raw keypoint array for the first detected person to the console.	OpenPose Python API examples. OpenCV documentation.	The script runs and shows a real-time video feed with a skeleton correctly tracking a person. Keypoint coordinates are printed to the console.
Fatima Mehdvi	1-7	**Literature Review & Exercise Finalization:** Begin a structured literature review on the biomechanics of upper-body therapeutic exercises. Focus on Shoulder Abduction, Elbow Flexion, and Neck movements. Confirm with the team that Shoulder Abduction and Elbow Flexion are the final choices for the MVP.	Physio-pedia, PubMed, academic journals.[22]	A summary of initial findings is documented. MVP exercises are confirmed and recorded in the project Wiki.
(R&D Specialist)	8-14	**Specification for Shoulder Abduction:** Create the first complete "Exercise Specification Document" for Shoulder Abduction. Identify the relevant OpenPose keypoints (BODY_25 model). Define the vectors and the exact	Biomechanics research.[29] OpenPose keypoint documentation.[7]	A complete, developer-ready specification document for Shoulder Abduction is committed to the GitHub Wiki.

Member	Day	Task Description	Key Resources / Actions	Deliverable / Success Criterion
		mathematical formula for calculating the abduction angle. Research and cite the clinically accepted normal range of motion.[26] Identify and define at least one common form error (e.g., shoulder hiking) and propose a method for its detection using keypoint data.		

Part III: System Architecture and MVP Development Roadmap

With the foundational environments validated, this section provides the master plan for constructing the Minimum Viable Product (MVP). It details the system's architecture, the development plan for each core module, and the precise data contracts that will ensure their seamless integration. This phase commences immediately following the successful completion of the two-week sprint.

3.1 The PhysioGuide System Architecture

The architecture of PhysioGuide is a classic real-time pipeline designed for low-latency feedback. It is composed of a processing node (the Python backend) and a presentation node (the Unity frontend), connected by a high-speed communication channel.

A high-level overview of the data flow is as follows:

1. **Input:** A standard webcam captures raw video frames of the user performing an exercise.
2. **Processing Node (Python Application):**
    - **Frame Capture:** An OpenCV module reads the video stream frame by frame.
    - **Pose Estimation:** Each frame is passed to the pyopenpose wrapper, which returns the 2D coordinates and confidence scores for all detected keypoints.
    - **Analysis Engine:** A custom module validates the keypoints using confidence score thresholds. It then uses the validated coordinates to calculate biomechanically relevant data, such as joint angles, based on the rules defined in the Exercise Specification Documents.
    - **Serialization & Broadcast:** The raw keypoints and calculated angles are packaged into a JSON object. A FastAPI WebSocket server then broadcasts this JSON payload over the network.

3. **Communication Channel:** A WebSocket connection provides a persistent, low-latency TCP/IP link between the server and the client.
4. **Presentation Node (Unity Application):**
    - **Reception & Deserialization:** A WebSocket-Sharp client receives the JSON payload. The data is deserialized into a structured C# object.
    - **Avatar Control:** The keypoint data is used to update the joint rotations of a virtual avatar, allowing the user to see a representation of their own movements in a virtual mirror.
    - **Feedback Logic:** The calculated angle data is consumed by a feedback manager, which compares it against the rules for the current exercise.
    - **Rendering:** Based on the feedback logic, the system renders visual cues (e.g., color changes on the avatar or UI elements) and triggers audio prompts to guide the user.
5. **Output:** The final, rendered scene is displayed to the user through the Oculus Quest 2's screens, and audio is delivered through its built-in speakers.

3.2 Module Development: The Pose Estimation Engine (Python/OpenPose)

- **Lead Developer:** Hussain Kazmi
- **Objective:** To create a robust, standalone Python application that encapsulates all pose estimation and analysis logic and exposes the results via a WebSocket server.

Development Steps:

1. **Structure the Application:** The "Hello, Pose" script from the initial sprint will be refactored into a more formal class-based structure. A main class, PoseProcessor, will manage the application lifecycle, including initializing the OpenPose wrapper, starting the video capture thread, and gracefully shutting down.
2. **Implement the Analysis Engine:** A separate module, analysis.py, will contain the functions for calculating joint angles. It will take the raw keypoint array and an exercise specification as input. This module must rigorously enforce the confidence score thresholding. If a required keypoint's confidence is too low, the angle calculation for that frame should return a null or invalid value, which will be propagated through the JSON payload.
3. **Integrate the WebSocket Server:** The FastAPI server code, initially prototyped by Taha, will be integrated into the main application. The primary processing loop will be:
    - while True:
        - Capture a frame from the webcam.
        - Process the frame with OpenPose.
        - If a person is detected, pass keypoints to the analysis engine.
        - Package the keypoints and calculated angles into the JSON data structure defined in Table 4.

        - Asynchronously broadcast the JSON string to all connected WebSocket clients.

3.3 Module Development: The Immersive Experience (Unity/C#)

- **Lead Developer:** Javeria Anwer
- **Objective:** To build the virtual gym environment, a simple user avatar that can be driven by the streamed pose data, and the UI for displaying feedback.

Development Steps:

1. **Build the VR Gym Environment:** Using the Unity Editor, create a simple, well-lit room. For the MVP, this can be constructed from basic geometric primitives (planes and cubes) to ensure high performance.[43] A critical element is a large virtual "mirror" placed in front of the player. This can be achieved using a second camera that renders its view to a Render Texture, which is then applied as the material on a large quad, creating a reflective surface.
2. **Create a Drivable Avatar:** A simple, low-polygon humanoid model (e.g., a stick-figure or basic robot) will be used for the MVP to maintain high frame rates. A C# script, AvatarController.cs, will be attached to the root of the avatar. This script will hold public references to the Transform components of the avatar's key joints (e.g., RightUpperArm, RightForearm, Torso). It will expose a public method, UpdatePose(PoseData data), which will be called by the WebSocket client upon receiving new data.
3. **Implement 2D-to-3D Pose Mapping:** The UpdatePose method will implement the simplified 2D-to-3D mapping strategy. A direct mapping of 2D screen coordinates to 3D world positions is complex and brittle. Instead, the system will use the *calculated angles* from the Python backend to drive the *joint rotations* of the 3D avatar. For example, the right_shoulder_abduction angle received in the data payload will be used to set the local rotation of the RightUpperArm GameObject in Unity. This abstraction provides a visually coherent representation of the user's pose without attempting a mathematically perfect but difficult 3D reconstruction.
4. **Implement the Feedback UI:** Create the necessary UI elements within the Unity scene. This will include TextMeshPro objects for displaying instructions ("Raise your arm to 90 degrees") and simple 3D meshes or UI panels that can have their material color changed programmatically (from a neutral color to green for correct or red for incorrect) by the feedback system.[1]

3.4 Module Development: The Real-Time Communication Bridge

- **Lead Developer:** Taha Khan
- **Objective:** To create a robust and reusable WebSocket client within Unity that can reliably receive and parse data from the Python backend without impacting application performance.

Development Steps:

1. **Create a Singleton Manager:** The "Hello, Network" script will be refactored into a persistent singleton pattern, creating a WebSocketManager.cs script. This ensures that there is only one instance of the WebSocket client managing the connection throughout the application's lifetime.

2. **Manage Threading and Data Synchronization:** This is a critical step. WebSocket client libraries, including WebSocket-Sharp, typically perform network operations on a background thread to avoid freezing the main application. However, the Unity Engine API is not thread-safe; any action that affects a GameObject or UI element (like updating the avatar's rotation or changing a UI color) *must* be executed on the main thread.[45] To solve this, the WebSocketManager will implement a thread-safe producer-consumer queue.
    - **Producer (Background Thread):** The ws.OnMessage event handler will fire on the background thread. Its only job will be to deserialize the incoming JSON string into a C# PoseData object and add it to a concurrent queue (e.g., ConcurrentQueue<PoseData>).
    - **Consumer (Main Thread):** The Update() method of the WebSocketManager, which is called by Unity every frame on the main thread, will check if the queue contains any data. If it does, it will dequeue the PoseData object and dispatch it to the rest of the application using a standard C# event (e.g., public static event Action<PoseData> OnPoseReceived).
3. **Implement Data Deserialization:** Define a set of C# classes that exactly mirror the structure of the JSON data contract (Table 4). A library like Newtonsoft.Json (available as a Unity package) will be used to automatically and efficiently deserialize the JSON strings into these C# objects.

3.5 Implementing the Core Feedback Loop

- **Lead Developers:** Javeria Anwer & Taha Khan
- **Objective:** To integrate the developed modules into a functional feedback system that responds to the user's movements in real time.

Integration Steps:

1. **Data Subscription:** The AvatarController.cs and a new FeedbackManager.cs script will both subscribe to the OnPoseReceived event from the WebSocketManager.
2. **Avatar Driving:** When the AvatarController receives a new PoseData object, it will use the calculated angles to update the rotations of the corresponding avatar joints, making the avatar in the mirror mimic the user.
3. **Rule Application and Feedback Triggering:** The FeedbackManager will also receive the PoseData. It will access the calculated angles and compare them against the thresholds and ranges defined in the active Exercise Specification Document (e.g., Table 3). Based on this comparison, it will call methods on the UI rendering components to change colors and trigger audio cues, thus closing the feedback loop.[1]

The following tables provide the critical data specifications required for parallel development and successful integration.

Table 3: Biomechanical Specification for MVP Exercises (Example: Shoulder Abduction)

Parameter	Specification
Exercise Name	Shoulder Abduction (Right Arm)
Key Joints (BODY_25)	RShoulder (2), RElbow (3), RHip (8)
Primary Angle	Angle between the torso and the upper arm.
Vector Definitions	v_torso = RHip - RShoulder; v_arm = RElbow - RShoulder
Angle Formula	angle = arccos(dot(normalize(v_torso), normalize(v_arm)))
Correct ROM	Target peak angle: 90° ± 10°.[27]
Common Error 1	**Shoulder Hiking:** Detected if RShoulder.y vertical position increases by >10% of torso length (RHip.y - Neck.y) during the abduction phase.
Common Error 2	**Leaning Torso:** Detected if the angle of v_torso relative to the image's vertical axis changes by > 15°.

Table 4: Python-Unity Data Transmission Contract (JSON Structure)

```
JSON
{
 "timestamp": 167761.12345,
 "persons":,
    "calculated_angles": {
      "right_shoulder_abduction": 85.3,
      "right_elbow_flexion": 170.1,
      "left_shoulder_abduction": 15.2,
      "left_elbow_flexion": 168.9
    }
  }
 ]
```

}

Part IV: Integration, Validation, and Future Trajectory

This final section outlines the strategy for assembling the individual modules into a cohesive application, defines a protocol for validating the MVP's functionality, and provides a strategic roadmap for development beyond the initial project scope.

4.1 A Phased Integration Strategy

A bottom-up, incremental integration strategy is recommended to manage complexity and simplify debugging. Each phase builds upon a previously validated foundation.

- **Phase 1: Network and Data Contract Validation:** This phase connects the full Python Pose Estimation Engine to the WebSocketManager in Unity. The primary goal is to verify that the complete JSON data packet, as defined in Table 4, is being successfully streamed from the Python backend and correctly deserialized by the Unity client. The success criterion is seeing the structured, deserialized data printed to the Unity console in real time without errors.
- **Phase 2: Avatar Driving and Pose Visualization:** This phase connects the WebSocketManager to the AvatarController. The objective is to drive the virtual avatar's movements using the streamed pose data. The success criterion is observing the virtual avatar in the VR gym mimic the gross movements of a person standing in front of the webcam. This phase validates the 2D-to-3D angle mapping logic.
- **Phase 3: Feedback Loop Activation:** The final integration phase connects the FeedbackManager to the data stream. The goal is to create a fully functional feedback loop. A team member will perform the specified exercises (Shoulder Abduction, Elbow Flexion) in front of the webcam. The success criterion is observing the correct visual (red/green color changes) and audio feedback being triggered within the VR environment in response to both correct and incorrect movements, as defined by the biomechanical rules.

4.2 MVP Validation Protocol

Once the MVP is integrated, it must be validated against its core objectives as outlined in the project proposal.[1] The validation protocol should focus on performance, accuracy, and usability.

- **Performance Testing (Latency):** End-to-end latency is the most critical performance metric. This is the time delay between a physical movement occurring and the corresponding feedback being rendered in VR. A simple method to measure this is to set up a high-speed camera to record both the user and the VR display (viewed on a monitor). By analyzing the video frame by frame, the team can count the number of frames between the initiation of a movement and the system's response, providing an approximate latency measurement.
- **Accuracy Testing (Angle Measurement):** The accuracy of the system's angle calculations must be verified. A physical goniometer can be used to manually measure a subject's joint angle as they hold a static pose (e.g., holding their arm at 90 degrees of abduction). This manual measurement can then be compared to the angle being calculated and displayed by the PhysioGuide system. A series of these tests at different angles will provide a measure of the system's accuracy and identify any systematic bias.

- **Usability Testing:** The "usability trials with students or volunteers" mentioned in the proposal should follow a structured script to gather qualitative feedback.[1] Key questions to answer include:
    - **Onboarding:** Can a new user start the application and understand the objective without assistance?
    - **Feedback Clarity:** Is the visual (red/green) and audio feedback intuitive and easy to understand?
    - **Comfort:** Is the VR experience comfortable? Are there any signs of motion sickness or disorientation?
    - **Engagement:** Do users find the experience more engaging than performing the exercises without guidance?

4.3 Beyond the MVP: Recommendations for Future Work

The successful completion of the MVP establishes a strong foundation. The following recommendations provide a strategic roadmap for expanding the project's capabilities, aligning with the long-term vision outlined in the proposal.[1]

- **Therapist Dashboard and Data Logging:** The most valuable extension is the creation of a therapist-facing dashboard. This requires adding a database (e.g., SQLite for simplicity, or a cloud-based solution like Firebase) to the Python backend. During each session, the system would log key metrics: timestamps, joint angles, form error occurrences, and repetition counts. A separate web application could then query this database, allowing a therapist to remotely review a patient's session history, track progress over time, and identify persistent issues.
- **Expanded and Customizable Exercise Library:** The current architecture, based on Exercise Specification Documents, is inherently modular. A future development phase should focus on creating a system where new exercises can be added by simply creating a new specification file (e.g., in JSON or XML format) without needing to recompile the core application. This would allow therapists to define and assign custom exercise routines to patients.
- **Enhanced and Instructive Feedback:** The MVP's binary red/green feedback is functional but limited. A more advanced system could provide more instructive feedback. For example, if a user is not raising their arm high enough, the system could render a virtual arrow pointing upwards or display a text prompt saying "Raise your arm higher." This requires a more sophisticated rules engine but would significantly enhance the system's therapeutic value.
- **Transition to 3D Sensing:** To overcome the fundamental limitations of 2D webcam tracking, the highest-impact technical upgrade would be to integrate a 3D depth sensor. This would involve:
    1. Replacing the OpenCV webcam input with an SDK for a device like an Intel RealSense or Azure Kinect.
    2. Utilizing a pose estimation model capable of directly inferring 3D keypoint coordinates (x, y, z).

3. Updating the Python analysis engine to perform vector math in 3D space, which would yield far more accurate and robust joint angle calculations, free from perspective distortion.
4. Enhancing the AvatarController in Unity to use this 3D data for a more precise positional and rotational mapping of the avatar's limbs.

By following this roadmap, the PhysioGuide project can evolve from a functional MVP into a comprehensive and clinically relevant telerehabilitation platform.